

FENIXCMS_9

FASE 9 - GATE DE PRODUCCION, CI/CD REAL, VALIDACION AUTOMATICA Y RELEASE SEGURO

Documento tecnico para preparar FenixCMS para produccion.

Repositorio: fenixcmssl-spec/moreno

Rama principal: main

Prioridad: CRITICA

Donde se ejecuta: AI Studio + GitHub. El despliegue al VPS solo debe realizarse despues de superar el Gate de Produccion.

0. DIAGNOSTICO ACTUAL

La auditoria actual de main despues de FENIXCMS_8 y del flujo reciente de activacion de cuentas confirma que FenixCMS ya contiene muchas defensas de runtime, auditorias y suites de pruebas, pero el repositorio no tiene todavia una pipeline principal que funcione como barrera obligatoria antes de produccion.

El workflow actualmente visible en .github/workflows esta orientado a generar los PDFs de la documentacion. No es un Gate completo de CI/CD.

El package.json ya dispone de comandos relevantes:

```
npm run lint
npm test
npm run audit:runtime
npm run audit:secrets
npm run check:production-env
npm run audit:production
npm run build
```

Tambien existe scripts/audit-production-source-of-truth.ts, pero el flujo de GitHub Actions no lo utiliza como comprobacion obligatoria de release.

.nvmrc fija Node 22 y existe package-lock.json, por lo que hay base para una instalacion reproducible mediante npm ci.

Conclusión de la auditoria: el proximo riesgo real es que un cambio futuro rompa una proteccion ya construida y aun asi llegue a main y pueda ser llevado al VPS.

1. OBJETIVO DE FENIXCMS_9

Convertir GitHub Actions en una barrera automatica de produccion.

Regla:

NO GREEN CI = NO RELEASE

NO RELEASE VALIDADO = NO DEPLOY PRODUCCION

Flujo:

Commit / Pull Request

|

v

CI completo

|

+-- npm ci

+-- Prisma validate

+-- Prisma generate

+-- PostgreSQL temporal

+-- migrations

+-- audit runtime

+-- audit secrets

+-- audit source-of-truth

+-- tests

+-- lint

+-- build

|

v

Release Candidate GREEN

|

v

Deploy manual/protegido

|

v

Smoke Tests

|

v

Produccion

2. RESULTADO ESPERADO

FENIXCMS_9 debe dejar implementado:

- CI para Pull Request.
- CI para main.
- Node fijado mediante .nvmrc.
- npm ci obligatorio.
- Prisma validate y generate.
- PostgreSQL temporal para integracion.
- prisma migrate deploy en la base de CI.
- audit:runtime.
- audit:secrets.
- audit:source-of-truth.
- suite completa de tests.
- lint.
- build.
- release manifest por commit SHA.
- artefacto de release.
- Gate manual para production.
- concurrency para impedir dos deploys simultaneos.
- smoke tests posteriores al deploy.
- procedimiento de rollback.
- comprobacion de salud liveness/readiness.
- no usar prisma migrate reset en produccion.

GitHub recomienda usar setup-node para fijar el runtime en Actions; la version actual del repositorio oficial es v7. GitHub tambien mantiene checkout v7 como version actual del action oficial.

Referencias verificadas:

<https://docs.github.com/actions/tutorials/build-and-test-code/nodejs>

<https://github.com/actions/setup-node/releases>

<https://github.com/actions/checkout>

3. FASE 9A - WORKFLOW PRINCIPAL DE CI

Crear:

.github/workflows/ci.yml

Triggers:

- pull_request contra main;
- push contra main;
- workflow_dispatch.

Permisos del job normal:

permissions:

contents: read

No otorgar contents: write al job que solo ejecuta validaciones.

4. FASE 9B - VERSION DE NODE

Utilizar .nvmrc como contrato del runtime.

Implementacion recomendada:

```
actions/checkout@v7
actions/setup-node@v7
node-version-file: .nvmrc
cache: npm
```

No utilizar node latest para el release de produccion.

El objetivo es que el mismo major version usado en local y VPS sea el usado en CI.

5. FASE 9C - REPRODUCIBILIDAD NPM

En CI y release usar:

```
npm ci
```

No utilizar npm install como instalacion de release.

El package-lock.json debe permanecer sincronizado con package.json.

Crear un check que falle si el lockfile no representa exactamente el package.json.

6. FASE 9D - CONTRATO DE NODE Y NPM

Añadir en package.json engines compatibles con la version realmente soportada por FenixCMS.

Ejemplo conceptual:

```
engines:
node: 22.x
npm: 10.x
```

El valor definitivo debe coincidir con las versiones que realmente se prueben en staging y VPS.

7. FASE 9E - VALIDACION DE PRISMA

Antes de tests:

```
npx prisma validate
npx prisma generate
```

Esto debe fallar el job si existe un error de schema.

8. FASE 9F - POSTGRESQL TEMPORAL DE CI

CI debe disponer de un PostgreSQL efimero.

Nunca utilizar la DATABASE_URL real del VPS.

Variables de CI, por ejemplo:

```
DATABASE_URL=postgresql://fenix_ci:fenix_ci@localhost:5432/fenixcms_ci
```

Despues:

```
npx prisma migrate deploy
```

Nunca:

```
prisma migrate reset
```

El esquema real debe poder desplegarse en una base limpia sin destruir datos.

9. FASE 9G - AUDITORIA DE FALLBACKS

Ejecutar obligatoriamente:

`npm run audit:runtime`

Debe bloquear la release si detecta regresiones como:

- Firebase en runtime;
- tenant_demo sin guarda;
- generacion local de licencias;
- segunda fuente de verdad;
- runtime demo.

10. FASE 9H - AUDITORIA DE SECRETOS

Ejecutar:

`npm run audit:secrets`

Debe fallar si aparecen:

- contraseñas maestras;
- DATABASE_URL en client components;
- PAYPAL_CLIENT_SECRET en cliente;
- tokens o secretos hardcodeados.

11. FASE 9I - SOURCE OF TRUTH

El repositorio ya contiene:

`scripts/audit-production-source-of-truth.ts`

Crear en package.json:

`audit:source-of-truth`

apuntando al script anterior.

CI debe ejecutar:

`npm run audit:source-of-truth`

Esto convierte la promesa de FENIXCMS_8 en una comprobación automática.

12. FASE 9J - COMPLETAR LA SUITE MAESTRA

`scripts/run-all-tests.ts` debe ser la puerta única de pruebas.

Incluir:

- tests de fases 1-8;
- account activation tests;
- production source-of-truth tests;
- production gate tests nuevos;
- PayPal webhook tests;
- tenant isolation tests;
- money precision tests;
- checkout tests.

Regla:

`npm test`

debe ejecutar todas las suites obligatorias.

13. FASE 9K - ACCOUNT ACTIVATION REGRESSION TEST

El flujo nuevo de activación de cuenta debe tener cobertura automática.

Casos obligatorios:

1. token valido;
2. token inexistente;
3. token expirado;
4. token ya utilizado;
5. segundo uso del mismo token;
6. usuario PENDING_ACTIVATION no puede login;
7. usuario pasa a ACTIVE despues de establecer password;
8. token persistido mediante hash;
9. password nunca aparece en logs;
10. logout y session revocation siguen funcionando.

14. FASE 9L - NO PASS FALSO

En ningun workflow se debe utilizar:

```
|| true  
continue-on-error: true
```

para ocultar un fallo de seguridad, test, lint, Prisma o build.

Solo pueden existir excepciones expresas para pasos informativos no bloqueantes.

15. FASE 9M - LINT

Ejecutar:

```
npm run lint
```

Si falla:

```
RELEASE = FAILED
```

No hacer commit automatico para esconder warnings o errores.

16. FASE 9N - BUILD

Ejecutar:

```
npm run build
```

Debe ser una puerta obligatoria.

Comprobar adicionalmente que el artefacto Next.js standalone esperado exista.

17. FASE 9O - DEPENDENCY AUDIT

El release debe ejecutar una comprobacion de dependencias runtime.

Como punto de partida:

```
npm audit --omit=dev
```

Definir una politica para no bloquear por vulnerabilidades no explotables de desarrollo, pero cualquier riesgo critico del runtime debe impedir la release hasta resolverse o documentarse formalmente.

18. FASE 9P - RELEASE MANIFEST

Generar:

```
release-manifest.json
```

Debe contener:

```
application  
commit  
node
```

npm
prisma
audits
tests
lint
build
createdAt

No incluir secretos.

El commit SHA del manifest debe coincidir con el artefacto creado.

19. FASE 9Q - ARTEFACTO DE RELEASE

Crear un artefacto con nombre:

fenixcms-release-\${GITHUB_SHA}

Debe poder identificarse sin ambigüedad.

El artefacto debe incluir como mínimo:

- build standalone;
- package-lock.json;
- release-manifest.json;
- información de commit;
- migraciones necesarias para deployment.

20. FASE 9R - RELEASE WORKFLOW

Crear:

.github/workflows/release.yml

Debe utilizar workflow_dispatch.

No desplegar automáticamente cada push a main.

El workflow debe:

1. descargar el artefacto CI;
2. verificar SHA;
3. verificar manifest;
4. comprobar environment production;
5. comprobar configuración;
6. ejecutar pre-deploy checks;
7. ejecutar migration segura;
8. desplegar release inmutable;
9. reiniciar servicio;
10. ejecutar smoke tests;
11. registrar PASS o rollback.

21. FASE 9S - GITHUB ENVIRONMENT PRODUCTION

Crear environment:

production

Secretos reales deben vivir en el environment de GitHub o en el mecanismo seguro elegido para el VPS.

Separar:

- CI;
- staging;
- production.

Nunca utilizar secretos de production en tests.

22. FASE 9T - CONCURRENCY

El release debe impedir dos deploys simultaneos.

Usar una concurrencia equivalente a:

```
group: fenixcms-production
cancel-in-progress: false
```

Objetivo: proteger la integridad del VPS y del estado de migraciones.

23. FASE 9U - SHA Y RELEASE VERIFICATION

El release debe comprobar que el SHA solicitado:

- pertenece a main;
- tiene CI verde;
- coincide con el artefacto;
- coincide con release-manifest.json.

Si no coincide:

DEPLOY = REJECTED

24. FASE 9V - PRODUCTION CONTRACT

Crear:

scripts/validate-production-contract.ts

Este script debe validar sin secretos reales el contrato estructural de produccion.

Debe comprobar como minimo:

- DATABASE_URL requerido;
- APP_URL requerido;
- APP_URL HTTPS;
- PAYPAL_CLIENT_ID requerido;
- PAYPAL_CLIENT_SECRET requerido;
- PAYPAL_WEBHOOK_ID requerido;
- PAYPAL_MODE correcto;
- PAYPAL_BASE_URL correcto para Live;
- callbacks no localhost;
- storage configurado;
- NODE_ENV=production para el release.

Si PAYPAL_MODE=live y PAYPAL_BASE_URL no apunta al endpoint Live correcto, el resultado debe ser FAIL, no warning.

25. FASE 9W - STAGING

Antes de production usar un environment staging separado.

Staging debe utilizar:

- PostgreSQL staging;
- PayPal Sandbox;
- storage staging;
- APP_URL staging.

No mezclar credenciales.

26. FASE 9X - PAYPAL SANDBOX E2E

Crear un Gate de aceptacion para una compra completa en PayPal Sandbox.

Debe demostrar:

1. crear Order;
2. approval real;
3. return;
4. capture;
5. webhook/reconciliation cuando corresponda;
6. Payment;
7. Tenant;
8. License;
9. Subscription;
10. Invoice;
11. acceso al CMS.

Un mock local NO cuenta como prueba Sandbox.

27. FASE 9Y - HEALTH CHECK

Usar los endpoints existentes:

/api/health?type=liveness

/api/health?type=readiness

Liveness debe confirmar proceso vivo.

Readiness debe confirmar PostgreSQL disponible.

Despues del deploy ambos deben devolver el estado esperado.

28. FASE 9Z - POST DEPLOY SMOKE TEST

Comprobar como minimo:

- homepage;
- health liveness;
- health readiness;
- login valido;
- login invalido;
- session;
- logout;
- storefront resolve;
- tenant isolation;
- endpoint billing protegido;
- webhook con firma invalida rechazado;
- account activation.

Un smoke test fallido implica rollback o bloqueo, nunca PASS.

29. FASE 9AA - RELEASE DIRECTORIES EN VPS

Utilizar releases inmutables, por ejemplo:

/opt/fenixcms/releases/<sha>/

/opt/fenixcms/current -> /opt/fenixcms/releases/<sha>

El servicio systemd debe apuntar a current.

No modificar directamente los archivos de una release activa.

30. FASE 9AB - SWITCH ATOMICO

No copiar archivo por archivo sobre la aplicacion en funcionamiento.

Secuencia:

descargar release

|

v

validar


```
|
v
preparar release
|
v
switch current
|
v
restart systemd
|
v
smoke tests
```

31. FASE 9AC - DATABASE MIGRATIONS

En production:

```
npx prisma migrate deploy
```

Nunca:

```
prisma migrate reset
```

Las migraciones deben diseñarse para que el código anterior y el nuevo puedan coexistir durante el cambio cuando exista riesgo de despliegue parcial.

Evitar cambios destructivos en una sola release si requieren downtime o si pueden romper el código anterior.

32. FASE 9AD - BACKUP

Si una release incluye cambios de schema o migraciones con riesgo de datos, exigir backup antes del deploy.

El pipeline puede detenerse esperando confirmación del backup.

No automatizar rollback destructivo de PostgreSQL.

33. FASE 9AE - ROLLBACK

Si los smoke tests fallan:

1. marcar release FAIL;
2. identificar release anterior;
3. volver current al SHA anterior;
4. reiniciar systemd;
5. comprobar liveness;
6. comprobar readiness;
7. registrar rollback.

La base de datos no debe retroceder automáticamente salvo que exista un mecanismo de rollback de datos expresamente probado.

34. FASE 9AF - BRANCH PROTECTION

Configurar GitHub para exigir como mínimo:

- CI PASS;
- review requerida;
- status checks requeridos;
- no force push;
- ramas protegidas.

El merge a main debe ser imposible mientras el CI obligatorio falle.

35. FASE 9AG - CI STATIC TEST

Crear:

tests/production-gate.test.ts

Debe comprobar como minimo:

- ci.yml existe;
- release.yml existe;
- package-lock existe;
- .nvmrc existe;
- package.json contiene scripts de audit;
- source-of-truth audit existe;
- health route existe;
- release manifest schema existe;
- no workflow usa secretos hardcodeados;
- no release workflow usa prisma migrate reset;
- CI utiliza npm ci;
- CI ejecuta npm test;
- CI ejecuta lint;
- CI ejecuta build.

36. FASE 9AH - DOCUMENTACION OPERATIVA

Crear:

docs/DEPLOY_PRODUCTION_FENIXCMS.md

Debe contener:

1. requisitos;
2. variables;
3. build;
4. migrations;
5. release;
6. smoke tests;
7. backup;
8. rollback;
9. systemd;
10. Nginx;
11. PostgreSQL;
12. recovery.

37. FASE 9AI - INFORME DE PRODUCCION

Generar un artifact:

production-readiness-report.json

Debe indicar:

commit
node
npm
prisma
runtimeAudit
secretsAudit
sourceOfTruthAudit
tests
lint
build
sandboxE2E
releaseCandidate

Sin secretos.

38. FASE 9AJ - PROMPT MAESTRO PARA AI STUDIO

Copiar este bloque completo en AI Studio.

INICIO FASE FENIXCMS_9

DONDE SE EJECUTA:

AI STUDIO / repositorio fenixcmssl-spec/moreno

OBJETIVO:

Crear el Gate obligatorio de produccion de FenixCMS.

Ningun commit puede ser considerado listo para VPS si no pasa CI completo.

REGLA:

NO GREEN CI = NO RELEASE

NO RELEASE VALIDADO = NO DEPLOY PRODUCCION

PASO 1 - AUDITORIA

Inspecciona .github/workflows, package.json, .nvmrc, package-lock.json, scripts, tests y prisma.

Enumera primero los huecos.

PASO 2 - CI

Crear .github/workflows/ci.yml con pull_request, push a main y workflow_dispatch.

Usar checkout@v7 y setup-node@v7 con node-version-file=.nvmrc y cache npm.

Usar permissions contents: read.

PASO 3 - POSTGRES

Añadir PostgreSQL temporal de CI.

Nunca utilizar la DB del VPS.

PASO 4 - VALIDACION

Ejecutar:

npm ci

npx prisma validate

npx prisma generate

npx prisma migrate deploy

npm run audit:runtime

npm run audit:secrets

npm run audit:source-of-truth

npm test

npm run lint

npm run build

Ningun fallo debe ocultarse.

PASO 5 - SOURCE OF TRUTH

Añadir audit:source-of-truth a package.json y ejecutarlo en CI.

PASO 6 - TESTS

Actualizar scripts/run-all-tests.ts para incluir las nuevas pruebas de FENIXCMS_8 y FENIXCMS_9 y el flujo de activation.

PASO 7 - ENVIRONMENT

Crear validate-production-contract.ts para validar configuracion estructural sin exponer secretos.

PASO 8 - RELEASE

Crear .github/workflows/release.yml con workflow_dispatch y environment production.

Verificar SHA y artefacto.

No desplegar cualquier SHA arbitrario.

PASO 9 - VPS

Usar release inmutable en /opt/fenixcms/releases/<sha> y symlink current.

Systemd debe apuntar a current.

PASO 10 - DB

Usar prisma migrate deploy.

Nunca migrate reset.

PASO 11 - HEALTH

Ejecutar liveness y readiness despues de deploy.

PASO 12 - SMOKE

Probar homepage, login, session, logout, storefront, tenant isolation, billing protegido, webhook invalido y account activation.

PASO 13 - ROLLBACK

Si smoke falla, volver al release anterior y comprobar health.

No hacer rollback destructivo de DB automaticamente.

PASO 14 - BRANCH PROTECTION

Documentar CI required y review required.

PASO 15 - PAYPAL SANDBOX

Mantener diferencia clara entre mock, sandbox y live.

La prueba E2E Sandbox debe quedar registrada.

PASO 16 - INFORME

Entregar archivos creados, workflows, scripts, tests, resultados exactos de npm ci, Prisma, audits, tests, lint, build, artefacto, smoke y blockers.

No escribir PASS si no se ejecuto realmente.

FIN FASE FENIXCMS_9

39. CRITERIOS DE ACEPTACION

[] ci.yml creado y ejecutandose en PR y main.

[] release.yml creado y protegido.

[] Node se obtiene de .nvmrc.

[] npm ci funciona.

[] Prisma validate PASS.

[] Prisma generate PASS.

[] migrations en PostgreSQL temporal PASS.

[] audit:runtime PASS.

[] audit:secrets PASS.

[] audit:source-of-truth PASS.

[] account activation tests PASS.

[] npm test PASS.

[] npm run lint PASS.

[] npm run build PASS.

[] dependency audit ejecutado.

[] release manifest generado.

[] artifact por SHA generado.

[] environment production configurado.

[] deploy manual/protegido.

[] no deploy concurrente.

[] migration production segura.

[] health liveness PASS.

[] health readiness PASS.

[] smoke tests PASS.

[] rollback documentado y comprobado.

[] PayPal Sandbox E2E PASS.

[] branch protection activa.

40. DECISION FINAL

FENIXCMS_9 no modifica el CMS funcional por capricho. Su objetivo es proteger todo lo que ya se ha construido en FENIXCMS_1 a FENIXCMS_8.

Una vez cerrada FENIXCMS_9, el proyecto debe pasar a una auditoria final y a un staging controlado antes de abrir produccion.

El criterio final es:

codigo validado

+

database migration validada

+

sandbox E2E

+

release identificable

+

smoke tests

+

rollback

=

candidato real a produccion

